

This architectural design prioritizes **security, scalability, and modularity**, ensuring the Banking Management System can handle sensitive financial data while providing a modern user experience.

1. UI Architecture

- **Framework:** Next.js 14+ (App Router) for Server-Side Rendering (SEO/Security) and optimized data fetching.
- **Language:** TypeScript (Strict Mode).
- **Rendering Strategy:**
 - * **Dashboard/Admin:** Client-side heavy with SSR for initial state.
 - * **Marketing/Public:** Static Site Generation (SSG) for performance.

2. Folder Structure (Feature-First)

src/	
app/	# App Router (Routes)
components/	# Shared UI (Button, Input, Modal)
features/	# Domain-specific logic
auth/	# Login, MFA, Password Reset
banking/	# Transfers, Bill Pay
dashboard/	# Analytics, Spending insights
loans/	# Application forms, Status tracker
hooks/	# Global hooks (useAuth, useMFA)
lib/	# Configs (Axios, Tailwind, i18n)
store/	# Zustand state management
types/	# Global TypeScript interfaces

3. Key Components

- **Data Grid:** TanStack Table (for employee/admin lists with sorting, filtering, and row actions).
- **Charts:** Recharts (for AI spending insights and trend analysis).
- **Forms:** React Hook Form + Zod (for schema-based validation).
- **Security:**
 - * `MFAInput``: OTP input component.
 - * `ProtectedLayout``: High-order wrapper that validates session and MFA status.

4. Routing & Authentication

- **Middleware:** Next.js Middleware checks session tokens and MFA status before rendering routes.

- **Auth Flow:**

1. **Login:** User enters credentials.
2. **MFA Challenge:** If valid, redirect to `/mfa-verify``.
3. **Completion:** Store encrypted JWT in `httpOnly`` cookie; verify on every server-side request.
4. **Role-Based Access Control (RBAC):** `AdminRoute`` vs `CustomerRoute`` components wrapper.

5. State Management

- **Zustand:** Lightweight global store for user sessions, account summaries, and temporary UI states (e.g., sidebar collapse, notification toasts).
- **TanStack Query (React Query):** Primary tool for API data. It handles **caching, revalidation, and loading states** for transactions and loan status, preventing stale financial data.

6. Forms & Validation

- **Validation:** Zod schemas shared between frontend and backend to ensure parity.
- **Logic:** React Hook Form for performance (minimizing re-renders on large banking forms).
- **Security:** Masked inputs for PII (Personally Identifiable Information) and Account Numbers.

7. Dashboard Layout

- **Shell Pattern:**

* **Sidebar:** Navigation, Quick Action buttons (Transfer Money).

* **Header:** Global Search (transactions), Notification Center, User Profile (quick settings).

* **Content Area:** Modular cards.

Customer: Balance overview, Spending chart, Quick Pay.

Employee: Pending loans table, Flagged transaction alerts.

8. UI Libraries

- **Styling:** Tailwind CSS (utility-first for rapid development).
- **UI Components:** Shadcn/UI (accessible, unstyled, radix-primitive base).
- **Icons:** Lucide React.
- **Internationalization:** `next-intl`` (for regulatory compliance and regional support).

Implementation Strategy for Key Requirements

| Requirement | Implementation Strategy |

| :--- | :--- |

| **MFA** | ``onUnauthorized`` redirect to MFA route; JWT token flagged ``isMFVerified: false``. |

| **Fraud Detection** | Real-time WebSockets to push fraud alerts to the Admin dashboard. |

| **Loan Approval** | Admin action triggers a React Query ``useMutation`` that invalidates the 'loans' cache, instantly updating the UI. |

| **Security** | ``Axios`` interceptors to inject CSRF tokens into every request; ``react-idle-timer`` to auto-logout users after 5 minutes of inactivity. |

Architectural Principles

1. **Immutability:** Financial data is never edited directly; operations are logged as events.
2. **Zero Trust:** No data is rendered on the client until the JWT is verified against the backend and the MFA claim is validated.
3. **Observability:** Integrated Sentry for error tracking and PostHog for auditing user navigation flow.